

Aufgabe 3

Bearbeitungszeit: vier Wochen (bis Freitag, 12. Juni 2026)

Mathematischer Hintergrund: Iterative Verfahren zur Lösung linearer Gleichungssysteme, dünnbesetzte Matrizen

Informatischer Hintergrund: dynamische Programmierung, *greedy*-Algorithmen

Elemente von C++: Klassen, Überladen von Operatoren

Ich empfehle Ihnen diesen Modus zur Nachahmung.
Schwerlich werden Sie je wieder direct eliminieren,
wenigstens nicht wenn Sie mehr als 2 Unbekannte haben.
Das indirecte Verfahren lässt sich halb im Schläfe ausführen,
oder man kann während desselben an andere Dinge denken.

(C. F. Gauß in einem Brief an C. L. Gerling vom 26.12.1823)

Während seinen Berechnungen stieß Gauß immer wieder auf Gleichungssysteme, die zu groß waren, als dass er sie mit der direkten Methode der Gauß-Elimination berechnen konnte. Darum entwickelte er um 1820 das wohl erste Iterationsverfahren, das Gauß-Seidel-Verfahren, das heute auch als Einzelschrittverfahren bekannt ist.

Iteratives Lösen linearer Gleichungssysteme

Beim iterativen Lösen linearer Gleichungssysteme handelt es sich um ein Approximationsverfahren zum Bestimmen der Lösung eines Gleichungssystems. Diese Verfahren stehen im Gegensatz zu den direkten Verfahren zum Lösen linearer Gleichungssysteme (z. B. Gaußsches Eliminationsverfahren, *LR*- oder *QR*-Zerlegung). Bei den direkten Verfahren kommt man nach endlich vielen Schritten zur *genauen* Lösung des Gleichungssystems, bei den iterativen Verfahren nähert man sich in jedem Schritt der Lösung.

Entwicklung iterativer Verfahren

Bei der Simulation technischer Prozesse treten an vielen Stellen sehr große lineare Gleichungssysteme auf. Einige Beispiele sind die Diskretisierung partieller Differentialgleichungen und die Computertomographie. In Zeiten ohne Rechner war es so gut wie unmöglich, mit dem direkten Verfahren größere Gleichungssysteme zu lösen, da diese enorm viel Rechenaufwand erfordern. Es war also notwendig, Verfahren zu entwickeln, mit denen man schneller an die Lösung kam.

In vielen der Anwendungsfälle treten lineare Gleichungssysteme mit einigen hunderttausend Unbekannten auf. Direkte Verfahren sind zwar für jede Dimension anwendbar, der benötigte Rechenaufwand ist aber von der Größenordnung $\mathcal{O}(n^3)$ und steigt daher mit der Dimension so stark an, dass man zur Lösung von Gleichungssystemen mit 10^5 , 10^6 oder noch mehr Unbekannten auch *große* Rechner *lange* beschäftigen kann.

Weiterhin benötigt man zum Beispiel zum Abspeichern der gesamten Matrix eines linearen Gleichungssystems mit 10^6 Unbekannten und Einträgen des Typs `double`¹ insgesamt

$$8 \cdot (10^6)^2 \text{ Byte} = 8 \cdot 10^{12} \text{ Byte} = 8000 \text{ GB} = 8 \text{ TB}$$

Speicherplatz. Soviel Arbeitsspeicher steht heute in der Regel nur auf Supercomputern zur Verfügung.

Dünnbesetzte Matrizen

Oft sind die Matrizen der linearen Gleichungssysteme in Anwendungsfällen nicht voll besetzt, das heißt sie enthalten sehr viele Nullen. Man bezeichnet üblicherweise eine Matrix als *dünn besetzt* (engl. *sparse*), wenn die Anzahl der von Null verschiedenen Einträge pro Zeile durch eine Konstante beschränkt ist, die wesentlich kleiner als die Spaltendimension der Matrix ist. Dünnbesetzte Matrizen erfordern sehr viel weniger Speicher, da die Nullen nicht mehr abgespeichert werden müssen.

Vorteile und Nachteile von Iterationsverfahren gegenüber direkten Verfahren

Je größer die Dimension des Gleichungssystems, desto schneller ist das Iterationsverfahren (meistens) im Vergleich zu den direkten Verfahren. Schon nach wenigen Schritten erhält man oft eine ziemlich genaue Lösung.

Durch die Rundungsfehler im Rechner – besonders bei schlecht konditionierten Matrizen – liefert das Iterationsverfahren sogar meist genauere Ergebnisse als das direkte Verfahren. Darum wurde auch die Methode der Nachiteration entwickelt. Bei dieser wird auf das Ergebnis des direkten Verfahrens ein iteratives Verfahren angewandt, um die Genauigkeit der Lösung zu erhöhen (siehe Lehrveranstaltung Numerische Analysis I).

Bei den direkten Verfahren (z.B. *LR*- oder *QR*-Zerlegung) zur Lösung eines linearen Gleichungssystems $Ax = b$ wird die Koeffizientenmatrix A im Laufe der Rechnung verändert. Ist die Dimension der dünn besetzten Matrix A sehr groß, so ist das veränderte A im allgemeinen nicht mehr ebenso dünn besetzt und nicht mehr im Hauptspeicher darstellbar. Direkte Verfahren sind daher oft nicht mehr möglich bzw. es ist ein sehr aufwendiger Datentransfer (z.B. zur Zwischenspeicherung der Daten auf Festplattenspeicher) nötig. Iterative Verfahren verändern die Koeffizientenmatrix hingegen nicht, sondern berechnen sukzessiv Näherungsvektoren für die Lösung und benötigen daher viel weniger Speicherplatz.

Auch der Nachteil iterativer Verfahren soll nicht verschwiegen werden: Man muss sicherstellen, dass sie konvergieren. Das ist oft nicht ganz einfach. Weiterhin muss man ein geeignetes Abbruchkriterium bereitstellen.

Aufgabenstellung

Mit Hilfe einer gut konzipierten Softwarebibliothek (engl. *library*) für Matrizen und Vektoren lassen sich iterative Verfahren zur Lösung von linearen Gleichungssystemen mit einem sehr geringem Programmieraufwand implementieren. Dies soll mittels Matrix- und Vektorklassen an einigen Beispielen erprobt werden. Ziel ist es, die Algorithmen so zu formulieren, dass sie im Wesentlichen mit Matrix-Vektor-Multiplikationen durchführbar sind.

Wir wollen hier zwei Verfahren behandeln. Diese sind

- a) das Gesamtschrittverfahren,
- b) das Verfahren der konjugierten Gradienten (CG-Verfahren) für symmetrische und positiv definite Matrizen.

¹Nach der Norm IEEE 754 (IEEE Standard for Floating-Point Arithmetic) belegt eine Fließkommavariablen doppelter Genauigkeit (engl. *double precision*; in C++ vom Typ `double`) 1 Bit für das Vorzeichen, 11 Bit für den Exponenten und 52 Bit für die Mantisse, so dass man zum Abspeichern insgesamt 64 Bit = 8 Byte Speicherplatz benötigt.

Sie sollten danach in der Lage sein, jederzeit weitere Algorithmen hinzuzufügen. Dies könnte etwa die *LR*-Zerlegung, die *QR*-Zerlegung oder ein weiteres iteratives Verfahren [7] sein.

Wir empfehlen Ihnen, `git` zur Versionsverwaltung benutzen, während Sie Ihren Code entwickeln. So können Sie sich bereits mit der Benutzung eines Versionskontrollsystems vertraut machen, das Sie, einmal gelernt, nicht nur beim Programmieren, sondern in vielen Bereichen Ihres Studiums unterstützen wird (z.B. bei der Versionierung von Texten für Seminararbeiten oder Ihrer Bachelor-Arbeit). In den folgenden Aufgaben wird die Verwendung von `git` auch in den Testaten geprüft werden, insofern sollten Sie den Umgang damit in der aktuellen Aufgabe bereits üben.

Vektor- und Matrixklasse

Um die Algorithmen dieser Aufgabe möglichst übersichtlich gestalten zu können, werden die Vektor- und Matrixoperationen in eigenen Klassen implementiert. Um die Programme besser kontrollieren zu können, sind die Schnittstellen der Klassen in den Header-Dateien `vector.h` und `sparse_matrix.h` fest vorgegeben. Die Implementierung der `Vector`-Klasse stellen wir Ihnen bereits in der Datei `vector.h` zur Verfügung². Ihre Aufgabe ist es, analog eine Klasse `SparseMatrix` für dünnbesetzte Matrizen mit den entsprechenden Operationen in einer neu zu erstellenden Datei `sparse_matrix.cpp` zu entwickeln.

Umsetzung dünnbesetzter Matrizen mit Hilfe einer Hashtabelle

Um eine dünnbesetzte Matrix abzuspeichern, sind viele Datenstrukturen denkbar [7]. In dieser Aufgabe soll eine *Hashtabelle* (engl. hash table, hash map) benutzt werden. Dabei handelt es sich wie bei den Klassen `set` und `map` aus der C++ Standard Library [1, 5] um einen *assoziativen Container*. In diesem kann auf die zugeordneten Daten oder *Werte* (engl. value) über einen *Schlüssel* (engl. key) zugegriffen werden. Daher stellt die Hashtabelle eine Abbildung (engl. map) dar.

Das Besondere an der Hashtabelle ist, dass bei ihr im Gegensatz zu einem Feld nicht für alle möglichen Schlüssel Speicherplatz reserviert werden muss. Gleichzeitig ist die Hashtabelle so konzipiert, dass der Aufwand für den Zugriff auf ein Element von der Größenordnung $\mathcal{O}(1)$, also konstant und unabhängig von der Anzahl der Einträge in der Hashtabelle ist.

Intern verwendet die Hashtabelle eine *Hashfunktion*, die sie auf einen Schlüssel anwendet. Der Funktionswert wird dann einer Position in einem Feld zugeordnet, an der die Werte abgespeichert werden sollen. Die *Hashfunktion* ist so konstruiert, dass möglichst wenig Schlüssel den gleichen Funktionswert liefern. Trotzdem ist dies nicht ganz zu vermeiden, da die Menge der möglichen Schlüssel viel größer ist als die Länge der internen Tabelle. Wird zwei Schlüssel nun doch die gleiche Stelle im internen Feld zugewiesen, so bezeichnet man dies als *Kollision* (engl. collision). Ein gängiger Weg zur Kollisionsbehandlung ist, an dieser Stelle im internen Feld einen Zeiger auf eine verkettete Liste abzulegen, in der dann die zugehörigen Daten abgelegt werden. Im Englischen wird diese Methode der Kollisionsbehandlung als *chaining* bezeichnet.

Die C++ Standard Library [1] ist eine Softwarebibliothek, die Bestandteil der Programmiersprache C++ ist und viele immer wieder benötigte Datenstrukturen und Algorithmen auf diesen Datenstrukturen bereitstellt. Diese Implementierungen sind auf Fehlerfreiheit getestet und vielfach bewährt. Seit der Aktualisierung der C++-Standardbibliothek auf Version C++11, ist mit der Klasse `std::unordered_map` bereits eine Hashtabelle vorhanden.³

Für die Verwendung solcher Container im Programm definiert man sich am einfachsten mittels `using` einen neuen Datentyp. Mit

```
#include <unordered_map>
using MyHash = std::unordered_map<std::string, int>;
```

²Da es sich hierbei um eine Template-Klasse handelt, befindet sich die Implementierung ebenfalls in der Header-Datei

³Benutzen Sie bitte die Compileroption `-std=c++17`, da die Testumgebung auch Funktionen des C++17-Standards nutzt.

wird `MyHash` als Datentyp Hashtabelle mit `std::string`-Schlüsseln und `int`-Daten deklariert. Damit kann z.B. ein Telefonregister realisiert werden:

```
MyHash telReg;

telReg["Ron"]      = 79825;
telReg["Hermine"]  = 22235;
telReg["Harry"]   = 40108;
std::cout << telReg["Hermine"] << std::endl;
```

Um die Elemente des Containers zu durchlaufen, werden wie gewohnt *Iteratoren* benutzt. Iteratoren sind eine Verallgemeinerung von Zeigern. Sie erlauben es, mit verschiedenen Containern auf gleiche Weise zu arbeiten. Die Ausgabe aller Elemente des Telefonregisters erfolgt z.B. über

```
for (MyHash::iterator it = telReg.begin(); it != telReg.end(); ++it)
    std::cout << "Name:_" << it->first << ",_TelNr:_" << it->second <<
        std::endl;
```

Dabei bedeutet `it->first` (gleichbedeutend mit `(*it).first`) den Zugriff auf den Schlüssel (in diesem Falle der Name) und `it->second` den Zugriff auf die Daten (hier die Telefonnummer), da das Tupel aus Schlüssel und Daten in einem Objekt `std::pair` abgelegt werden. Beachten Sie, dass der Zugriff auf ein Element über den `operator[]` dieses Element erzeugt und in die Hashtabelle aufnimmt, falls es noch nicht existiert. Möchten Sie nur testen, ob ein Schlüssel existiert, benutzen Sie die Methode `find()`. Sei dem C++11-Standard gibt es sogenannte *range-based for-loops*, die auch für Hashtabellen benutzt werden können. Der obige Code zur Ausgabe des Telefonregisters sieht dann wie folgt aus:

```
for (std::pair<std::string, int> entry : telReg)
    std::cout << "Name:_" << entry.first << ",_TelNr:_" << entry.second
        << std::endl;
```

Um die Indizes eines Eintrags der Matrix abzulegen, kann auch die Klasse `std::pair` aus der Standardbibliothek

```
using coord_t = std::pair<std::size_t, std::size_t>;
```

verwendet werden. Ein solches Paar ist dann der Datentyp für die Schlüssel der Hashtabelle der dünnbesetzten Matrix. Zusätzlich muss nun eine Hashfunktion für den neuen Datentyp `coord_t` in Form einer Klasse bereitgestellt werden. Dabei kann man auf die vordefinierten Hashfunktionen für Standarddatentypen zurückgreifen:

```
struct coord_hash {
    std::size_t operator()(coord_t key) const {
        std::hash<size_t> size_t_hash;
        return size_t_hash(key.first) + size_t_hash(key.second);
    }
};
```

Mit

```
using HashMap = std::unordered_map<coord_t, double, coord_hash>;
```

erzeugt man nun den Datentypen der für die dünnbesetzte Matrix nötigen Hashtabelle.

Anforderungen an die Matrixklasse

Die Schnittstelle der Matrixklasse ist in der Header-Datei `sparse_matrix.h` festgelegt und darf *nicht* verändert werden, um eine problemlose Interaktion mit der Testumgebung `test_mv.o` und der Praktikums Umgebung `unit.o` zu gewährleisten. An Ihre Matrixklasse werden einige Anforderungen gestellt. Durch

```
SparseMatrix A(m,n);
```

wird eine dünnbesetzte Matrix **A** mit **m** Zeilen und **n** Spalten angelegt, die nur Nullen enthält. Um auch Felder von dünnbesetzten Matrizen anlegen zu können (dabei kann der Konstruktor nur ohne Parameter aufgerufen werden), sollten **m** und **n** den Default-Wert 1 bekommen. Der Lesezugriff auf das Matricelement A_{ij} (auch für `const SparseMatrix &A`) erfolgt mittels

```
A(i,j) oder A.Get(i,j),
```

der Schreibzugriff über

```
A.Put(i,j,x).
```

Besonders wichtig ist, zu vermeiden, dass in der Hashtabelle ein Eintrag gespeichert wird, der den Wert Null hat.

Falls die Matrix das gewünschte Element aufgrund der Dimension nicht enthält, sollte mit einer Fehlermeldung abgebrochen werden. Die Dimension der Matrix sollte sich analog zur Elementfunktion `GetLength()` der Vektorklasse über die beiden Elementfunktionen `GetRows()` und `GetCols()` auslesen lassen. Mit

```
A.Redim(m,n);
```

wird die Matrix neu dimensioniert und wieder auf die Nullmatrix gesetzt. Der (private) Datenbereich Ihrer Klasse hat die folgende Form:

```
std::size_t rows_, cols_; // Matrixdimension
HashMap mat_;             // Hashmap fuer Matricelemente
```

Der Konstruktor sollte dann die Dimensionen in den Attributen `rows_` und `cols_` richtig setzen. Ihre Funktionen müssen dafür sorgen, dass die Matricelemente korrekt in der Hashtabelle `mat_` abgelegt werden.

Neben den üblichen Operatoren für Matrizen sollte Ihre Matrixklasse auch entsprechende Operatoren für das Matrix-Vektor-Produkt (Ax und $A^T x$) zur Verfügung stellen.

Test der Vektor- und Matrixklassen

Die Datei `test_mv.o` soll Ihnen bei der Fehlersuche in den Klassen helfen. Dazu müssen Sie aus Ihren Dateien für die Vektor- und Matrixklasse die Objektdateien `vector.o` und `sparse_matrix.o` kompilieren und diese mit `test_mv.o` zu einem ausführbaren Programm linkern. Durch den Aufruf dieses Programmes können Sie dann testen, ob Ihre Klassen das Richtige tun.

Das erfolgreiche Absolvieren der Tests heißt aber nicht, dass alles richtig implementiert ist. Verlassen Sie sich daher nicht blind darauf!

Darüber hinaus sollen Sie mithilfe der `MapraTest`-Klasse eigene Unit-Tests für die `SparseMatrix`-Klasse schreiben.

Gesamtschrittverfahren

Das Gesamtschrittverfahren wurde 1845 von Carl Gustav Jacobi entwickelt und man bezeichnet es daher auch als Jacobi-Verfahren. Löst man in einem linearen Gleichungssystem $Ax = b$ mit Matrix $A \in \mathbb{R}^{n \times n}$ die i -te Gleichung nach x_i auf, so erhält man das Gesamtschrittverfahren. Zerlegt man A in $A = A_L + D + A_R$, wobei A_L eine strikte untere Dreiecksmatrix und A_R eine strikte obere Dreiecksmatrix und D eine Diagonalmatrix sind, so lässt sich die so gewonnene Gleichung schreiben als $x = D^{-1}(b - (A_L + A_R)x)$. Wegen der Zerlegung der Matrix A in die Dreiecksmatrizen A_L und A_R und die Diagonalmatrix D fällt das Gesamtschrittverfahren unter die Splitting-Verfahren [7].

Die Multiplikation eines Vektors mit der Inversen einer Diagonalmatrix D bewirkt, dass jede Komponente des Vektors durch das entsprechende Diagonalelement geteilt wird. In der Praxis wird D nicht als Matrix sondern als Vektor d mit $D = \text{diag}\{d_1, \dots, d_n\}$ gespeichert. Die Operation $D^{-1}y$ lässt sich dann mit dem **operator**/ als Vektor-Vektor-Operation y/d in der Vektorklasse implementieren. Das Gesamtschrittverfahren für $Ax = b$ ist in Algorithmus 1 wiedergegeben.

Algorithmus 1 Gesamtschrittverfahren (Jacobi-Verfahren)

- 1: Gegeben seien $A \in \mathbb{R}^{n \times n}$ streng diagonaldominant und ein beliebiger Startvektor $x^{(0)} \in \mathbb{R}^n$.
 - 2: **for** $k = 0, 1, 2, \dots$ **do**
 - 3: $x^{(k+1)} \leftarrow (b - (A_L + A_R)x^{(k)})/d$
 - 4: Prüfe Abbruchbedingung.
 - 5: **end for**
-

Bleibt noch zu diskutieren, unter welchen Voraussetzungen das Verfahren konvergiert und welche Abbruchbedingung verwendet werden soll. Es gibt ein Konvergenzkriterium, das sich auch einfach überprüfen lässt:

Satz 1 (Hinreichende Bedingung für die Konvergenz des Gesamtschrittverfahrens) *Das Gesamtschrittverfahren konvergiert für jeden Startwert $x^{(0)} \in \mathbb{R}^n$, falls die Matrix A streng diagonaldominant ist, falls also gilt $|a_{ii}| > \sum_{1 \leq j \leq n, j \neq i} |a_{ij}|$ für $1 \leq i \leq n$.*

Die Iteration soll abgebrochen werden, wenn eine maximale Anzahl von Schritten k_{max} überschritten wird oder das Residuum kleiner als ein vorgegebenes $\varepsilon > 0$ wird.

In der praktischen Umsetzung soll $A_L + A_R$ in der Matrix A gespeichert werden. Ist eine neue Näherung der Lösung berechnet, so kann die alte Lösung gelöscht werden, es ist also unnötig und Verschwendung, alle Iterationsvektoren zu speichern. Das Residuum lässt sich einfach berechnen, wenn man $b - (A_L + A_R)x^{(k)}$ zwischenspeichert.

Streng genommen muss zwischen Residuenvektor $r^{(k)} = b - Ax^{(k)}$ und Residuum $\|r^{(k)}\|_2$ unterschieden werden. Beachten Sie, dass ein kleines Residuum nicht zwangsweise bedeutet, dass eine gute Näherungslösung vorliegt. Überprüfen Sie, ob das Konvergenzkriterium aus Satz 1 erfüllt ist, bevor Sie die Iteration beginnen.

Konjugierte Gradienten – das CG-Verfahren

Das Verfahren der konjugierten Gradienten oder CG-Verfahren (engl. conjugate gradient method) für symmetrisch positiv definite (spd) Matrizen A ist eines der effizientesten Verfahren zur Lösung linearer Gleichungssysteme $Ax = b$ überhaupt. Es wurde 1952 von Hestenes und Stiefel veröffentlicht [4] und fällt in die Klasse der Krylov-Teilraumverfahren [6, 7]. Das CG-Verfahren ist in Algorithmus 2 formuliert.

Für eine symmetrisch positiv definite Matrix $A \in \mathbb{R}^{n \times n}$ ist $\langle \cdot, \cdot \rangle_A : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}, (x, y) \mapsto x^T A y$ ein Skalarprodukt und wird als Energieskalarprodukt bezeichnet. Folglich ist $\|x\|_A := \sqrt{\langle x, x \rangle_A}$ eine Norm auf $\mathbb{R}^n$, die sogenannte Energienorm.

Es bleibt wieder zu klären, unter welchen Voraussetzungen das Verfahren konvergiert und welches Abbruchkriterium sinnvoll ist. Es gilt der

Algorithmus 2 CG-Verfahren

```
1: Gegeben seien  $A \in \mathbb{R}^{n \times n}$  symmetrisch positiv definit und ein beliebiger Startvektor  $x^{(0)}$ .
2:  $r^{(0)} \leftarrow b - Ax^{(0)}$ ,  $d^{(0)} = r^{(0)}$ 
3: for  $k = 0, 1, 2, \dots$  do
4:    $\alpha_k \leftarrow \|r^{(k)}\|_2^2 / \langle Ad^{(k)}, d^{(k)} \rangle$   $\triangleright Ad^{(k)}$  für später speichern!
5:    $x^{(k+1)} \leftarrow x^{(k)} + \alpha_k d^{(k)}$ 
6:    $r^{(k+1)} \leftarrow r^{(k)} - \alpha_k Ad^{(k)}$ 
7:    $\beta_k \leftarrow \|r^{(k+1)}\|_2^2 / \|r^{(k)}\|_2^2$   $\triangleright \|r^{(k+1)}\|_2^2$  für später speichern!
8:    $d^{(k+1)} \leftarrow r^{(k+1)} + \beta_k d^{(k)}$ 
9:   Prüfe Abbruchbedingung.
10: end for
```

Satz 2 (Exakte Lösung beim CG-Verfahren) Für eine symmetrische positiv definite Matrix $A \in \mathbb{R}^{n \times n}$ findet das CG-Verfahren (unter der Annahme exakter Rechnung) nach höchstens n Schritten die exakte Lösung.

Wenn wir also voraussetzen, dass die Matrix symmetrisch und positiv definit ist, dann konvergiert das CG-Verfahren für jeden beliebigen Startwert. Meist möchte man aber nicht so viele Iterationen berechnen. Darüber hinaus lässt sich die Konvergenzgeschwindigkeit des CG-Verfahrens abschätzen:

Satz 3 (Fehlerabschätzung für das CG-Verfahren) Unter obigen Voraussetzungen gilt die Fehlerabschätzung

$$\|x - x^{(k)}\|_A \leq 2 \left(\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \right)^k \|x - x^{(0)}\|_A,$$

wobei $\kappa_2(A)$ die Kondition der Matrix A bezüglich der Euklidischen Norm und x die exakte Lösung bezeichnet.

Die Beweise der Sätze findet man in [2, 6, 7, 3]. In der Praxis ist diese Abschätzung als Abbruchkriterium wenig geeignet, da die Kondition $\kappa_2(A)$ unbekannt und nur aufwendig zu berechnen ist. Wichtig ist hingegen festzustellen, dass die Geschwindigkeit der Konvergenz des Verfahrens stark steigt, wenn die Kondition der Matrix sinkt.

Die Iteration soll hier wieder abgebrochen werden, sobald eine maximale Anzahl von Schritten k_{max} überschritten oder das Residuum kleiner als ein vorgegebenes $\varepsilon > 0$ wird.

Der Name konjugierte Gradienten erklärt sich wie folgt: Die Minimierung des quadratischen Funktionals $\Phi(x) = \frac{1}{2}x^T Ax - x^T b$ führt auf die notwendige Bedingung $\text{grad } \Phi(x) = Ax - b = 0$ (A symmetrisch und positiv definit ist dann hinreichend für ein Minimum). Wenn man $\Phi(x)$ nun schrittweise mittels des Ansatzes $x^{(k+1)} = x^{(k)} + \alpha_k d^{(k)}$ minimiert, so sind mit obiger Wahl die $d^{(k)}$ bzgl. A konjugiert, d.h.: $\langle d^{(k+1)}, d^{(k)} \rangle_A = (d^{(k+1)})^T A d^{(k)} = 0$.

Da wir ein einheitliches Abbruchkriterium vorgegeben haben, können beide Verfahren folgendermaßen implementiert werden:

```
int mapra::GSV (mapra::SparseMatrix& A, const mapra::Vector<double>& b
    , mapra::Vector<double>& x0, const int k_max, double& eps);
int mapra::CG(const mapra::SparseMatrix& A, const mapra::Vector<double
    >& b, mapra::Vector<double>& x0, const int k_max, double& eps);
```

Beim Gesamtschrittverfahren darf die Matrix A modifiziert werden, muss aber beim Verlassen der Funktion (Unterprogramm) wieder wie beim Eingang besetzt sein. Es brauchen und sollen auch keine weiteren Matrizen oder Felder (engl. arrays) von Vektoren angelegt werden. Beim Aufruf steht in `eps` die gewünschte Genauigkeit (Residuum). Beim Rücksprung soll `eps` die erreichte Genauigkeit enthalten. In der Funktion sollen die Voraussetzungen geprüft (gsv: A diagonaldominant, cg: A symmetrisch) und eine Warnung ausgegeben werden, wenn sie nicht erfüllt sind. Der Rückgabewert der Funktion ist die Anzahl der durchgeführten

Schritte (falls >0) bzw. ein Fehlercode. Dabei sollen folgende Fehler berücksichtigt werden: 0: `k_max` Iterationen erreicht (auch in diesem Fall soll aber `eps` die erreichte Genauigkeit enthalten), -1: Dimensionen von `A`, `b` und `x0` passen nicht zusammen.

Programmabnahme

Schreiben Sie zuerst eine `SparseMatrix`-Klasse, welche alle Tests erfüllt, die durch `test_mv` definiert sind. Lösen Sie dann die obige Aufgabe, sodass alle Tests der Praktikums Umgebung erfolgreich durchlaufen, und laden Sie anschließend Ihren Quellcode (als Gruppe) im Lernraum hoch. Danach lassen Sie Ihren Code testen. Um das Testat zu bestehen, sollten mindestens folgende Punkte erfüllt sein:

- Ihr Programm kompiliert und unsere vorgegebenen Tests werden bestanden.
- Ihr Code ist lesbar und wartbar (siehe dazu die Refactoring-Aufgabe).
- Sie haben eigene Tests geschrieben, mit denen die Funktionalität Ihres Codes überprüft wird, dies betrifft vor allem die `SparseMatrix`-Klasse.

Literatur

- [1] *C/C++-Referenz*. <http://www.cppreference.com/index.html>.
- [2] DAHMEN, W. und A. REUSKEN: *Numerische Mathematik für Ingenieure und Naturwissenschaftler*. Springer Verlag, Heidelberg, 2. Auflage, 2008.
- [3] GOLUB, G. und C. VAN LOAN: *Matrix Computations*. Johns Hopkins University Press, Baltimore, 2003.
- [4] HESTENES, M. R. und E. STIEFEL: *Methods of Conjugate Gradients for Solving Linear Systems*. Journal of Research of the National Bureau of Standards, 49(6):409–436, 1952.
- [5] KUHLLINS, S. und M. SCHADER: *Die C++-Standardbibliothek*. Springer Verlag, Heidelberg, 2005.
- [6] MEISTER, A.: *Numerik linearer Gleichungssysteme*. Vieweg, Wiesbaden, 2005.
- [7] SAAD, Y.: *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, 2. Auflage, 2007. Die 1. Auflage ist online verfügbar unter <http://www-users.cs.umn.edu/~saad/books.html>.